



Group Members

Student 1: Gaurav Acharya — [MT26MCS024]
Student 2: Sandeep — [MT26MCS032]

Guide Name

Dr. A. S. Mokhade, sir

Practical Title:

CSP502 Software Lab-I:

Practical 3: Evidence-Based Debugging and Runtime Analysis

Problem Statement:

Concept: From Finding Bugs to Establishing Why Programs Fail

Part A: DEBUGGING COMMAND REFERENCE

EXECUTION CONTROL

- run [args] : Initiate program execution (with optional command-line arguments).
- continue : Resume execution after halting at a breakpoint.

BREAKPOINT MANAGEMENT

- break <target> : Suspend program execution at a designated location to inspect state.
- delete <id> : Remove a breakpoint no longer in use.

CODE STEPPING

- next : Execute the subsequent line without entering function calls.
- step : Step directly into a function.
- finish : Execute until the current function returns.

STATE INSPECTION

- print <var> : Evaluate the current value of variables/arrays.
- backtrace : Display the call stack to trace execution history.
- frame <id> : Navigate to a specific stack frame.
- up / down : Move up or down the call stack.

MEMORY ANALYSIS

- x/4x <address> : Directly examine raw values and data stored at a specific memory address.

COMPILATION SETUP

- gcc -g : Compile source code with embedded debugging symbols.
- gcc -fsanitize=address -g : Enable AddressSanitizer to detect memory boundary errors.

CRASH DIAGNOSTICS

- ulimit -c unlimited : Configure the OS to generate core dumps upon a crash.
- core-file <core> : Open a core dump for post-mortem analysis of the failure state.

MEMORY PROFILING

- valgrind ./prog : Identify memory leaks and incorrect memory access during runtime.

REFLECTION (RQ1)

Which information was easier to obtain from documentation than from experimentation, and which became meaningful only after actually using the debugger?

MT26MCS024 - Gaurav Acharaya:

While the official documentation effectively clarified the basic syntax and intended purpose of core commands (such as `break`, `next`, `step`, and `backtrace`), their practical mechanics remained abstract. The dynamic ways in which breakpoints alter execution flow, and how stack frames shift during operation, only became intuitive through hands-on application in GDB. This practical exposure was essential for making the debugging process comprehensible.

MT26MCS032 - Sandeep:

Similarly, foundational command usage was easily acquired by referencing the documentation. However, the theoretical concepts behind variables, stack frames, and program state transitions were heavily dependent on experimentation. The true value and meaning of these diagnostic commands were only realized when actively deployed to troubleshoot a live program.

Part D: Debugging Evidence Record

Program	Observed Symptom	Failure/Problem Location	Debugger Evidence	Root Cause	Correction	Verification
A	Prints 1, 2, 3, followed by an unexpected stack garbage value.	program01.c:6 (printf("%d\n", arr[i]));	print i showed i = 3 while valid array indices are 0 to 2. print arr[3] fetched uninitialized stack memory.	Off-by-one error in the loop condition (i <= 3 instead of i < 3).	Change loop condition to for (int i = 0; i < 3; i++).	Program prints 1, 2, 3 precisely and exits cleanly without out-of-bounds reads.
B	Program crashes instantly with a floating-point exception (SIGFPE).	program02.c:5 (printf("Result = %d\n", a / b));	backtrace showed termination at line 5 due to SIGFPE. print b confirmed b == 0.	Division by zero (a / b where divisor b is initialized to 0).	Ensure divisor b is non-zero or initialize b to a valid non-zero integer.	Program executes successfully and outputs the correct division result without crashing.
C	Program crashes instantly with a segmentation fault (SIGSEGV).	program03.c:4 (*p = 5;)	backtrace indicated crash at line 4. print p showed p = 0x0 or a wild uninitialized address.	Dereferencing an uninitialized wild pointer p.	Allocate memory via malloc or point p to a valid variable address before assignment.	Program executes without crashing and prints 5 correctly.
D	Runs for short names; crashes or corrupts heap when given names ≥20 chars; leaks memory on early exit.	records.c:9 (strcpy(s->name, name);) and main (malloc before argc check).	AddressSanitizer reported heap-buffer-overflow on write of size 1 at strcpy. GDB showed heap metadata corruption.	Unbounded string copy (strcpy) into a fixed buffer and premature memory allocation before validation.	Replace strcpy with strncpy, ensure null-termination, and move malloc after argument validation.	Tested with long inputs and invalid arguments; sanitizers and execution confirm safe truncation and proper cleanup.

Part E: Post-Mortem Debugging (Core Dump Analysis)

- **Termination Point:** Using `gdb ./programD core`, the backtrace pointed to `__GI__fortify_fail` or the memory boundary check failure instruction immediately following the heap buffer overflow.
- **Call Sequence Reconstruction:** `main()` $\rightarrow$ `process_record()` $\rightarrow$ `update_record()` $\rightarrow$ `strcpy()` (where heap corruption triggered an abort signal).
- **Additional GDB Investigation:** Allowed inspection of registers, stack frames, heap pointers (`print *student`), local variables, and raw memory bytes surrounding the buffer (`x/32bx s`) to measure overflow extent.
- **Limitations:** Post-mortem core dumps provide only a static snapshot at the exact moment of failure. Dynamic timing, user interaction sequences, and intermediate state transitions prior to the crash cannot be re-simulated from the dump alone.

Part F: gdb versus AddressSanitizer

- **Sanitizer Findings:** AddressSanitizer instantly pinpointed heap buffer overflows and uninitialized memory/leaks with exact stack traces, specifying whether the violation was a read or write and the exact byte offset.
- **GDB Role:** GDB provides interactive state inspection, stepping through execution flow, modifying variables on the fly, navigating stack frames, and performing post-mortem core dump analysis.
- **Can Sanitizers Replace Debuggers?** No. Sanitizers excel at detecting *that* a memory error occurred and *where*, but they do not explain *why* the business logic produced that state or how variables interacted over time.
- **When to Use Both:** Use AddressSanitizer during development and testing phases for fast, automated memory error detection; use GDB when deep logic inspection, core dump post-mortem analysis, or interactive step-through reasoning is required.
- **Source Code vs. Runtime Evidence:** Static claims regarding the presence of `strcpy` and buffer sizing were established from source code alone. However, proving that this triggers a heap corruption crash under specific input lengths, and demonstrating the exact memory layout failure, required runtime evidence (ASan and GDB).
- **Core-Dump Relation:** Core-dump analysis demonstrates that static code review only reveals *potential* vulnerabilities, whereas runtime execution evidence (and core dumps) reveals the actual execution reality and exact failure state.

Part G: AI as a Debugging Hypothesis Generator (Program D)

Aspect	AI Diagnosis	Runtime Evidence	Your Conclusion
Problem Identified	Flagged potential buffer overflow with strcpy and memory leak on early return.	AddressSanitizer and GDB confirmed a heap buffer overflow on strcpy and a memory leak when argc != 3.	AI correctly identified static vulnerabilities visible from source code syntax.
Failure Mechanism	Suggested strcpy writes beyond fixed buffer size.	GDB/ASan showed heap metadata corruption leading to SIGABRT when string length exceeded 20 bytes.	Validated; runtime evidence proved the exact mechanism of heap corruption.
Location	Identified lines containing strcpy(s->name, name) and malloc.	Stack trace pointed directly to update_record at strcpy and main at malloc.	AI location was accurate.
Assumptions Made	Assumed standard inputs could exceed buffer size and that checks were missing.	Verified through test cases with strings >20 characters.	Assumptions were valid and confirmed by runtime execution.
Proposed Correction	Suggested strncpy or snprintf and moving malloc after argument checks.	Recompiling with these fixes passed all sanitizers and test inputs.	AI's code recommendation was sound and effective.

Source Code vs. Runtime Evidence: Static claims regarding the presence of `strcpy` and buffer sizing were established from source code alone. However, proving that this triggers a heap-corruption crash under specific input lengths, and demonstrating the exact memory-layout failure, required runtime evidence (ASan and GDB).

Core-Dump Relation: Core-dump analysis demonstrates that static code review only reveals *potential* vulnerabilities, whereas runtime execution evidence (and core dumps) reveals the actual execution reality and exact failure state.

1. Discussion and Reflection: Gaurav Acharya

- **On Tool Discovery and Experimentation:** While the official documentation effectively clarified the basic syntax and intended purpose of core debugging commands (like `break`, `next`, and `backtrace`), their practical mechanics initially remained abstract. I found that the dynamic ways in which breakpoints alter execution flow, and how stack frames shift during operation, only became intuitive through hands-on application in GDB.
- **Symptom versus Root Cause:** This practical heavily reinforced that the location where a program fails is often completely different from where the underlying problem originated. A crash is usually just a side effect of an earlier mistake. For example, in Program D, a heap corruption crash only surfaced at the end of the program's execution, even though the root

cause—an unbounded `strcpy`—happened much earlier in the call stack.

- **Debugging in an AI-Assisted Environment:** AI tools are incredibly fast at acting as hypothesis generators and can identify theoretical flaws by examining source code alone. However, if an AI identifies a faulty line without running the program, it has merely generated a plausible diagnosis. You must use runtime tools, such as AddressSanitizer and GDB, to actually prove that the AI's theoretical bug results in a crash under specific conditions.
- **Engineering Responsibility:** Software engineers bear 100% of the responsibility for establishing code correctness before deployment. Even if AI-generated code passes initial tests or looks plausible, it will inevitably contain subtle bugs (like off-by-one errors or memory leaks) that fail in edge cases.

2. Discussion and Reflection: Sandeep

- **On Theory vs. Practical Debugging:** Foundational command usage was easily acquired by referencing the documentation. However, the theoretical concepts behind variables, stack frames, and program state transitions were heavily dependent on active experimentation. The true value and meaning of diagnostic commands were only realized when deployed to troubleshoot a live, faulty program.
- **Failure without Crash:** Testing Program D demonstrated that a program running without crashing is not guaranteed to be correct or safe. Silent vulnerabilities, such as buffer overflows, can still exist and corrupt memory in the background. This completely invalidates the dangerous assumption that "a successful run equals a correct program".
- **The Role of Sanitizers vs. Debuggers:** While AddressSanitizer is excellent at highlighting exactly what memory rule was broken and where, it cannot replace interactive debuggers. Tools like GDB are necessary to pause the program, inspect variables, and investigate the underlying logic flaws that led to the broken rule.
- **Production Scenarios:** In real-world, large-scale systems, crashes cannot always be reproduced immediately. If a production service fails, it is critical to preserve core dumps. These dumps provide a vital post-mortem snapshot of the program's memory, registers, and call stack at the exact moment it died, allowing us to establish why the program failed without needing to rerun it.